

03_monthly_salience_polling_merge

May 11, 2026

1 03 Monthly Salience-Polling Merge

This notebook merges monthly media salience indicators with monthly AfD Bundestag polling support from DAWUM.

Scope notes: - Descriptive only (no causal claims). - DAWUM-based merged panel coverage is 2017-01 to 2025-12. - Cologne 2016 is outside this DAWUM panel unless a separate historical polling supplement is added later.

```
[1]: from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

pd.set_option('display.max_columns', 200)
pd.set_option('display.width', 180)

PROJECT_ROOT = Path.cwd()
if PROJECT_ROOT.name == 'notebooks':
    PROJECT_ROOT = PROJECT_ROOT.parent

DATA_DIR = PROJECT_ROOT / 'data'
INDICATORS_DIR = DATA_DIR / 'indicators'
POLLING_DIR = DATA_DIR / 'polling' / 'processed'
PANEL_DIR = DATA_DIR / 'panel'
FIGURES_DIR = PROJECT_ROOT / 'figures' / '03_notebook_figures'

for d in [PANEL_DIR, FIGURES_DIR]:
    d.mkdir(parents=True, exist_ok=True)

MEDIA_PARQUET_PATH = INDICATORS_DIR / 'monthly_media_salience_indicator.parquet'
MEDIA_CSV_PATH = INDICATORS_DIR / 'monthly_media_salience_indicator.csv'

POLL_PARQUET_PATH = POLLING_DIR / 'afd_polling_monthly_2017_2025.parquet'
POLL_CSV_PATH = POLLING_DIR / 'afd_polling_monthly_2017_2025.csv'

PANEL_CSV_PATH = PANEL_DIR / 'monthly_salience_polling_panel.csv'
```

```

PANEL_PARQUET_PATH = PANEL_DIR / 'monthly_salience_polling_panel.parquet'

ANALYSIS_START = pd.Timestamp('2017-01-01')
ANALYSIS_END = pd.Timestamp('2025-12-31')
EXPECTED_MONTHS = pd.period_range(ANALYSIS_START, ANALYSIS_END, freq='M')

CHEMNITZ_MONTH = pd.Timestamp('2018-08-01')
CORRECTIV_MONTH = pd.Timestamp('2024-01-01')

print('PROJECT_ROOT:', PROJECT_ROOT)
print('MEDIA_PARQUET_PATH:', MEDIA_PARQUET_PATH)
print('POLL_PARQUET_PATH:', POLL_PARQUET_PATH)
print('PANEL_CSV_PATH:', PANEL_CSV_PATH)
print('FIGURES_DIR:', FIGURES_DIR)

```

```

PROJECT_ROOT: c:\PythonProjects\AfD-project
MEDIA_PARQUET_PATH: c:\PythonProjects\AfD-
project\data\indicators\monthly_media_salience_indicator.parquet
POLL_PARQUET_PATH: c:\PythonProjects\AfD-
project\data\polling\processed\afd_polling_monthly_2017_2025.parquet
PANEL_CSV_PATH: c:\PythonProjects\AfD-
project\data\panel\monthly_salience_polling_panel.csv
FIGURES_DIR: c:\PythonProjects\AfD-project\figures\03_notebook_figures

```

```

[2]: def load_table(preferred_parquet: Path, fallback_csv: Path) -> pd.DataFrame:
    if preferred_parquet.exists():
        return pd.read_parquet(preferred_parquet)
    if fallback_csv.exists():
        return pd.read_csv(fallback_csv)
    raise FileNotFoundError(f'Missing both parquet and csv: {preferred_parquet}\n
↳| {fallback_csv}')

```

```

media = load_table(MEDIA_PARQUET_PATH, MEDIA_CSV_PATH)
polling = load_table(POLL_PARQUET_PATH, POLL_CSV_PATH)

print('media shape:', media.shape)
print('polling shape:', polling.shape)
print('media columns:', list(media.columns))
print('polling columns:', list(polling.columns))
print('media head:')
print(media.head(3))
print('polling head:')
print(polling.head(3))

```

```

media shape: (156, 17)
polling shape: (108, 15)
media columns: ['month', 'year_month', 'year', 'month_num',
'nexis_total_results', 'media_salience_volume_raw',

```

```
'media_salience_volume_log1p', 'media_salience_volume_z', 'raw_article_count',
'download_fraction', 'retrieval_mismatch_flag', 'high_download_sample_flag',
'kept_article_count', 'cleaned_sample_size', 'cleaned_sample_fraction',
'cleaned_sample_log1p', 'cleaned_sample_z']
polling columns: ['month', 'year_month', 'month_dt', 'year', 'month_num',
'afd_poll_support_mean', 'afd_poll_support_median', 'afd_poll_support_min',
'afd_poll_support_max', 'afd_poll_support_sd', 'n_polls', 'n_pollsters',
'first_poll_date', 'last_poll_date', 'source']
```

media head:

```
      month year_month  year  month_num nexis_total_results
media_salience_volume_raw media_salience_volume_log1p media_salience_volume_z
raw_article_count \
0 2013-01 2013-01 2013          1          42
42          3.761200          -0.909941          42
1 2013-02 2013-02 2013          2          55
55          4.025352          -0.824250          55
2 2013-03 2013-03 2013          3          67
67          4.219508          -0.745150          67
```

```
      download_fraction retrieval_mismatch_flag high_download_sample_flag
kept_article_count cleaned_sample_size cleaned_sample_fraction
cleaned_sample_log1p cleaned_sample_z
0          1.0          False          False
38          38          0.904762          3.663562
-0.973925
1          1.0          False          True
41          41          0.745455          3.737670
-0.813715
2          1.0          False          True
61          61          0.910448          4.127134
0.254350
```

polling head:

```
      month year_month  month_dt  year  month_num afd_poll_support_mean
afd_poll_support_median afd_poll_support_min afd_poll_support_max
afd_poll_support_sd n_polls \
0 2017-01 2017-01 2017-01-01 2017          1          12.186667
12.0          11.0          14.5          1.139465          15
1 2017-02 2017-02 2017-02-01 2017          2          10.356522
10.0          8.0          13.0          1.374715          23
2 2017-03 2017-03 2017-03-01 2017          3          9.529167
9.2          7.0          11.5          1.408508          24
```

```
      n_pollsters first_poll_date  last_poll_date source
0          7 2017-01-11 2017-01-31 12:00:00 DAWUM
1          10 2017-02-01 2017-02-26 12:00:00 DAWUM
2          9 2017-03-01 2017-03-29 00:00:00 DAWUM
```

```

[3]: def ensure_month_keys(df: pd.DataFrame, name: str) -> pd.DataFrame:
    out = df.copy()

    if 'year_month' in out.columns:
        out['year_month'] = out['year_month'].astype(str)
    elif 'month' in out.columns:
        out['year_month'] = out['month'].astype(str)
    else:
        raise ValueError(f'{name}: missing both year_month and month columns')

    if 'month_dt' in out.columns:
        out['month_dt'] = pd.to_datetime(out['month_dt'], errors='coerce')
    else:
        out['month_dt'] = pd.to_datetime(out['year_month'] + '-01',
        ↪errors='coerce')

    out['year_month'] = out['month_dt'].dt.strftime('%Y-%m')
    out['month'] = out['year_month']

    if out['month_dt'].isna().any():
        bad = out.loc[out['month_dt'].isna(), ['year_month']].head(10)
        raise ValueError(f'{name}: month_dt parsing failed for rows: {bad}.
        ↪to_dict(orient="records")}')

    if out['year_month'].duplicated().any():
        dupes = out.loc[out['year_month'].duplicated(), 'year_month'].tolist()
        raise ValueError(f'{name}: duplicate month keys: {dupes[:10]}')

    out = out.sort_values('month_dt').reset_index(drop=True)
    return out

media = ensure_month_keys(media, 'media')
polling = ensure_month_keys(polling, 'polling')

print('media coverage:', media['month_dt'].min(), 'to', media['month_dt'].max())
print('polling coverage:', polling['month_dt'].min(), 'to', polling['month_dt'].
    ↪max())

if media['month_dt'].min().to_period('M') > ANALYSIS_START.to_period('M') or
    ↪media['month_dt'].max().to_period('M') < ANALYSIS_END.to_period('M'):
    raise ValueError('Media salience does not cover required analysis window
    ↪months 2017-01 to 2025-12')

polling_window = polling[(polling['month_dt'] >= ANALYSIS_START) &
    ↪(polling['month_dt'] <= ANALYSIS_END)].copy()
if polling_window.empty:

```

```

        raise ValueError('Polling data has no rows inside analysis window 2017-01-01
        to 2025-12-01')

```

```

poll_months = pd.PeriodIndex(polling_window['month_dt'].dt.to_period('M').
    unique(), freq='M')
missing_poll_months = EXPECTED_MONTHS.difference(poll_months)
print('Polling months in window:', len(poll_months))
print('Missing polling months in window:', [m.strftime('%Y-%m') for m in
    missing_poll_months])

```

```

media coverage: 2013-01-01 00:00:00 to 2025-12-01 00:00:00
polling coverage: 2017-01-01 00:00:00 to 2025-12-01 00:00:00
Polling months in window: 108
Missing polling months in window: []

```

```

[4]: media_window = media[(media['month_dt'] >= ANALYSIS_START) & (media['month_dt'] <=
    ANALYSIS_END)].copy()
media_window = media_window.sort_values('month_dt').reset_index(drop=True)

if len(media_window) != len(EXPECTED_MONTHS):
    raise ValueError(f'Media window rows mismatch: expected {len(EXPECTED_MONTHS)}, got {len(media_window)}')

media_months = pd.PeriodIndex(media_window['month_dt'].dt.to_period('M').
    unique(), freq='M')
missing_media_months = EXPECTED_MONTHS.difference(media_months)
if len(missing_media_months) > 0:
    raise ValueError('Missing media months in analysis window: ' + ', '.join(m.
        strftime('%Y-%m') for m in missing_media_months))

print('media_window shape:', media_window.shape)
print('media_window range:', media_window['month_dt'].min(), 'to',
    media_window['month_dt'].max())

```

```

media_window shape: (108, 18)
media_window range: 2017-01-01 00:00:00 to 2025-12-01 00:00:00

```

```

[5]: panel = polling.merge(
    media_window,
    on='year_month',
    how='inner',
    suffixes=('', '_media'),
    validate='one_to_one',
)

# Harmonize month/date columns after merge.
if 'month_dt' not in panel.columns and 'month_dt_media' in panel.columns:
    panel['month_dt'] = panel['month_dt_media']

```

```

if 'month' not in panel.columns and 'month_media' in panel.columns:
    panel['month'] = panel['month_media']

panel['month_dt'] = pd.to_datetime(panel['month_dt'], errors='coerce')
panel['month'] = panel['month_dt'].dt.strftime('%Y-%m')
panel['year_month'] = panel['month']

if panel['year_month'].duplicated().any():
    dupes = panel.loc[panel['year_month'].duplicated(), 'year_month'].tolist()
    raise ValueError(f'Panel has duplicate month keys: {dupes[:10]}')

if len(panel) != len(EXPECTED_MONTHS):
    raise ValueError(f'Panel rows mismatch: expected {len(EXPECTED_MONTHS)}, got {len(panel)}')

if panel['afd_poll_support_mean'].isna().any():
    raise ValueError('Missing afd_poll_support_mean after merge')

if panel['nexis_total_results'].isna().any():
    raise ValueError('Missing nexis_total_results after merge')

if 'retrieval_mismatch_flag' in panel.columns:
    mismatch_count = int(panel['retrieval_mismatch_flag'].fillna(False).
        astype(bool).sum())
    if mismatch_count != 0:
        raise ValueError(f'retrieval_mismatch_flag has {mismatch_count} True values in merged panel')

panel = panel.sort_values('month_dt').reset_index(drop=True)
print('panel shape:', panel.shape)
print('panel range:', panel['month_dt'].min(), 'to', panel['month_dt'].max())

```

panel shape: (108, 32)
panel range: 2017-01-01 00:00:00 to 2025-12-01 00:00:00

```

[6]: def zscore(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors='coerce')
    mu = s.mean()
    sd = s.std(ddof=0)
    if pd.isna(sd) or sd == 0:
        return pd.Series(np.zeros(len(s)), index=s.index, dtype=float)
    return (s - mu) / sd

panel['afd_poll_support_z'] = zscore(panel['afd_poll_support_mean'])

if 'media_salience_volume_z' not in panel.columns:
    panel['media_salience_volume_z'] = zscore(panel['nexis_total_results'])

```

```

if 'media_salience_volume_log1p' not in panel.columns:
    panel['media_salience_volume_log1p'] = np.log1p(pd.
    ↳to_numeric(panel['nexis_total_results'], errors='coerce'))

panel['month_index'] = np.arange(len(panel), dtype=int)

print('Added descriptive variables: afd_poll_support_z, month_index')
print(panel[['year_month', 'afd_poll_support_mean', 'afd_poll_support_z',
    ↳'media_salience_volume_z', 'month_index']].head())

```

```

Added descriptive variables: afd_poll_support_z, month_index
  year_month  afd_poll_support_mean  afd_poll_support_z  media_salience_volume_z
month_index
0    2017-01                12.186667             -0.521216                0.494077
0
1    2017-02                10.356522             -0.909724                0.388611
1
2    2017-03                 9.529167             -1.085357                0.388611
2
3    2017-04                 9.064286             -1.184043             -0.006887
3
4    2017-05                 8.258621             -1.355072             -0.059620
4

```

```

[7]: def add_shock_lines(ax):
    ax.axvline(CHEMNITZ_MONTH, color='#b22222', linestyle='--', linewidth=1.2,
    ↳alpha=0.85, label='Chemnitz (2018-08)')
    ax.axvline(CORRECTIV_MONTH, color='#1f4e79', linestyle='--', linewidth=1.2,
    ↳alpha=0.85, label='Correctiv (2024-01)')

    # a) AfD monthly polling support
    fig, ax = plt.subplots(figsize=(12, 5))
    ax.plot(panel['month_dt'], panel['afd_poll_support_mean'], color='#3b2f2f',
    ↳linewidth=2)
    add_shock_lines(ax)
    ax.set_title('AfD Bundestag Polling Support by Month (Descriptive)')
    ax.set_xlabel('Month')
    ax.set_ylabel('AfD polling support (%)')
    ax.grid(alpha=0.25)
    ax.legend(loc='upper left')
    fig.tight_layout()
    out1 = FIGURES_DIR / 'afd_poll_support_monthly.png'
    fig.savefig(out1, dpi=150)
    plt.show()
    print('Saved:', out1)

```

```

# b) Media salience over time
fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(panel['month_dt'], panel['nexis_total_results'], color='#2f5d50',
        linewidth=2)
add_shock_lines(ax)
ax.set_title('Monthly Nexis Total Results (Media Salience Volume)')
ax.set_xlabel('Month')
ax.set_ylabel('Nexis total results')
ax.grid(alpha=0.25)
ax.legend(loc='upper left')
fig.tight_layout()
out2 = FIGURES_DIR / 'media_salience_monthly_nexis_total.png'
fig.savefig(out2, dpi=150)
plt.show()
print('Saved:', out2)

# c) Standardized co-movement
fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(panel['month_dt'], panel['afd_poll_support_z'], color='#6a1b1a',
        linewidth=2, label='AfD polling z-score')
ax.plot(panel['month_dt'], panel['media_salience_volume_z'], color='#1a3d6a',
        linewidth=2, label='Media salience z-score')
add_shock_lines(ax)
ax.set_title('Descriptive Co-Movement: AfD Polling vs Media Salience_
        (Standardized)')
ax.set_xlabel('Month')
ax.set_ylabel('Standardized value (z)')
ax.grid(alpha=0.25)
ax.legend(loc='upper left')
fig.tight_layout()
out3 = FIGURES_DIR / 'descriptive_comovement_standardized.png'
fig.savefig(out3, dpi=150)
plt.show()
print('Saved:', out3)

# d) Number of polls per month
fig, ax = plt.subplots(figsize=(12, 4.5))
ax.plot(panel['month_dt'], panel['n_polls'], color='#4d4d4d', linewidth=1.8)
add_shock_lines(ax)
ax.set_title('DAWUM Bundestag Poll Coverage per Month')
ax.set_xlabel('Month')
ax.set_ylabel('n_polls')
ax.grid(alpha=0.25)
ax.legend(loc='upper left')
fig.tight_layout()
out4 = FIGURES_DIR / 'n_polls_per_month.png'
fig.savefig(out4, dpi=150)

```

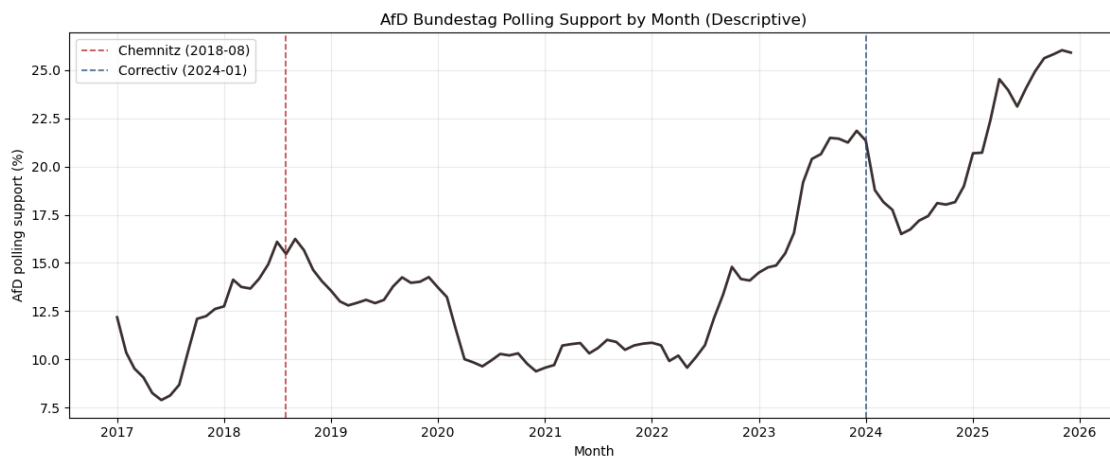


```

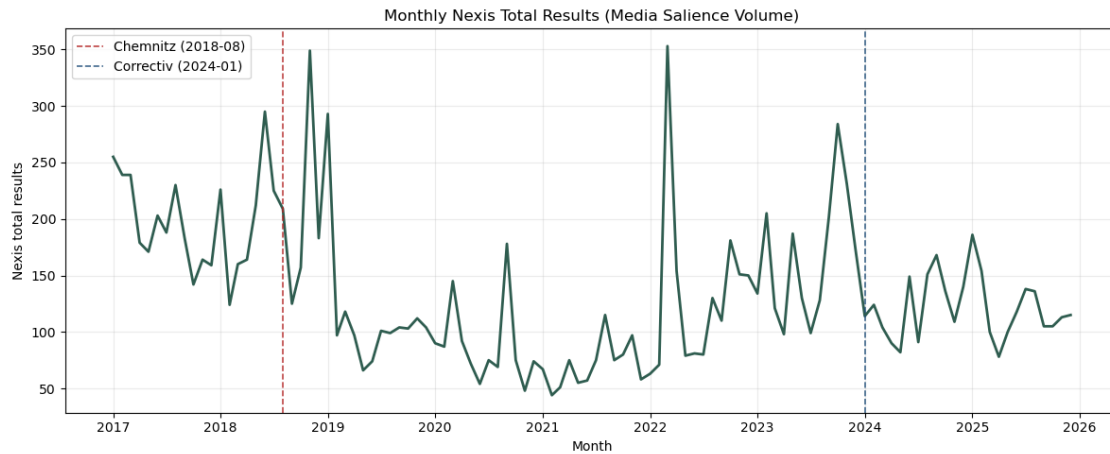
plt.show()
print('Saved:', out4)

# Optional e) Scatter descriptive association
fig, ax = plt.subplots(figsize=(6.5, 5.5))
ax.scatter(panel['media_salience_volume_z'], panel['afd_poll_support_z'],
           alpha=0.7, color='#274c77', edgecolor='white', linewidth=0.3)
ax.set_title('Descriptive Association: Media Salience vs AfD Polling_
           (Standardized)')
ax.set_xlabel('media_salience_volume_z')
ax.set_ylabel('afd_poll_support_z')
ax.grid(alpha=0.2)
fig.tight_layout()
out5 = FIGURES_DIR / 'scatter_media_vs_polling_z.png'
fig.savefig(out5, dpi=150)
plt.show()
print('Saved:', out5)

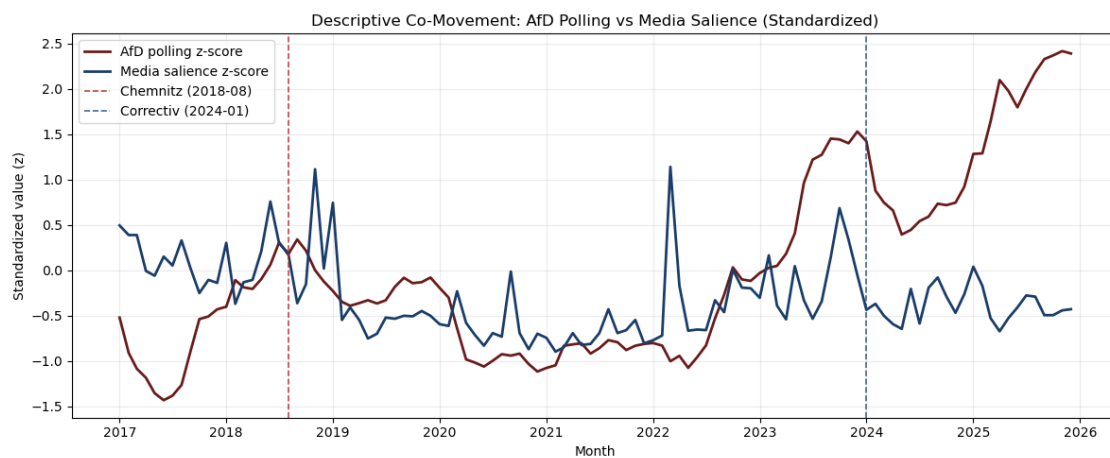
```



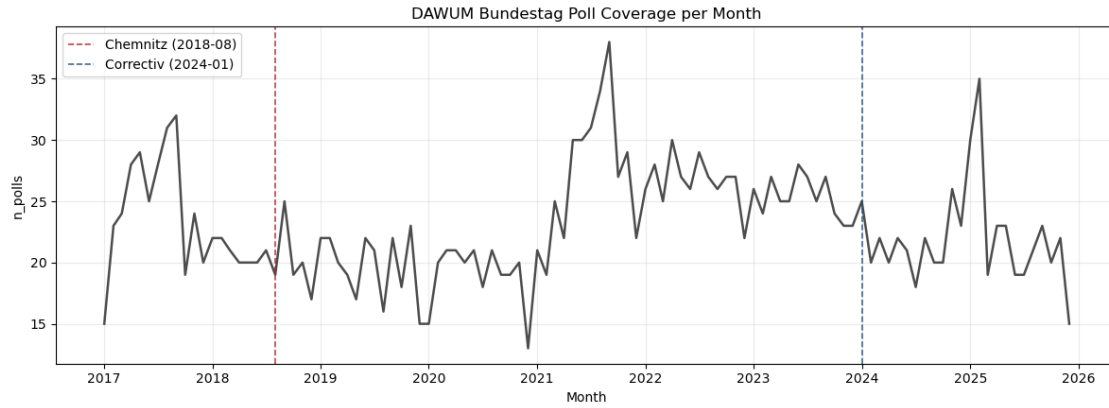
Saved: c:\PythonProjects\AfD-
project\figures\03_notebook_figures\afd_poll_support_monthly.png



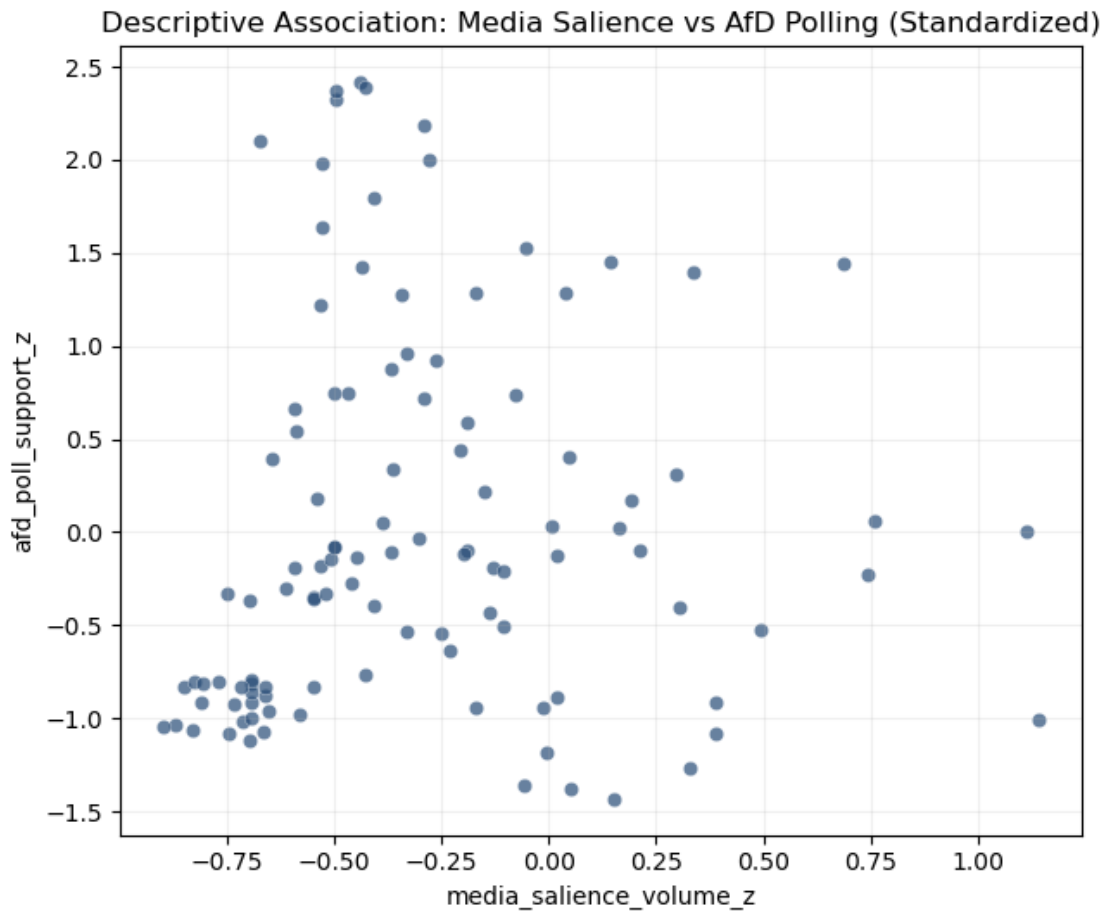
Saved: c:\PythonProjects\AfD-project\figures\03_notebook_figures\media_salience_monthly_nexis_total.png



Saved: c:\PythonProjects\AfD-project\figures\03_notebook_figures\descriptive_comovement_standardized.png



Saved: c:\PythonProjects\AfD-project\figures\03_notebook_figures\n_polls_per_month.png



Saved: c:\PythonProjects\AfD-

project\figures\03_notebook_figures\scatter_media_vs_polling_z.png

```
[8]: print('Panel date range:', panel['month_dt'].min(), 'to', panel['month_dt'].
      ↪max())
      print('Panel row count:', len(panel))

      print('AfD polling support summary:')
      print(panel['afd_poll_support_mean'].describe())

      print('Nexis salience summary:')
      print(panel['nexis_total_results'].describe())

      print('Top 10 months by Nexis total results:')
      print(panel.nlargest(10, 'nexis_total_results')[['year_month',
      ↪'nexis_total_results', 'media_salience_volume_z', 'afd_poll_support_mean',
      ↪'n_polls']])

      chem_window = panel[(panel['month_dt'] >= pd.Timestamp('2018-05-01')) &
      ↪(panel['month_dt'] <= pd.Timestamp('2018-11-01'))]
      cor_window = panel[(panel['month_dt'] >= pd.Timestamp('2023-10-01')) &
      ↪(panel['month_dt'] <= pd.Timestamp('2024-04-01'))]
      cols = ['year_month', 'nexis_total_results', 'media_salience_volume_z',
      ↪'afd_poll_support_mean', 'n_polls']

      print('Chemnitz +/- 3 months window:')
      print(chem_window[cols])
      print('Correctiv +/- 3 months window:')
      print(cor_window[cols])
```

Panel date range: 2017-01-01 00:00:00 to 2025-12-01 00:00:00

Panel row count: 108

AfD polling support summary:

count	108.000000
mean	14.641958
std	4.732661
min	7.896000
25%	10.723103
50%	13.745238
75%	17.511364
max	26.022727

Name: afd_poll_support_mean, dtype: float64

Nexis salience summary:

count	108.000000
mean	134.787037
std	64.178788
min	44.000000
25%	89.250000
50%	118.000000

```

75%      168.750000
max      353.000000
Name: nexis_total_results, dtype: float64
Top 10 months by Nexis total results:
   year_month  nexis_total_results  media_salience_volume_z
afd_poll_support_mean  n_polls
62    2022-03                    353          1.140057
9.920000          25
22    2018-11                    349          1.113690
14.645000          20
17    2018-06                    295          0.757742
14.930000          20
24    2019-01                    293          0.744559
13.568182          22
81    2023-10                    284          0.685234
21.437500          24
0     2017-01                    255          0.494077
12.186667          15
1     2017-02                    239          0.388611
10.356522          23
2     2017-03                    239          0.388611
9.529167           24
82    2023-11                    231          0.335878
21.239130          23
7     2017-08                    230          0.329286
8.687097           31
Chemnitz +/- 3 months window:
   year_month  nexis_total_results  media_salience_volume_z
afd_poll_support_mean  n_polls
16    2018-05                    212          0.210636
14.185000          20
17    2018-06                    295          0.757742
14.930000          20
18    2018-07                    225          0.296328
16.095238          21
19    2018-08                    209          0.190862
15.463158          19
20    2018-09                    125         -0.362836
16.244000          25
21    2018-10                    157         -0.151903
15.657895          19
22    2018-11                    349          1.113690
14.645000          20
Correctiv +/- 3 months window:
   year_month  nexis_total_results  media_salience_volume_z
afd_poll_support_mean  n_polls
81    2023-10                    284          0.685234
21.437500          24

```

82	2023-11	231	0.335878
21.239130	23		
83	2023-12	172	-0.053029
21.847826	23		
84	2024-01	114	-0.435344
21.340000	25		
85	2024-02	124	-0.369427
18.775000	20		
86	2024-03	104	-0.501260
18.159091	22		
87	2024-04	90	-0.593543
17.750000	20		

```
[9]: preferred_front = [
    'month', 'year_month', 'month_dt', 'year', 'month_num',
    'afd_poll_support_mean', 'afd_poll_support_median', 'afd_poll_support_sd',
    'n_polls', 'n_pollsters',
    'nexis_total_results', 'media_salience_volume_raw',
    'media_salience_volume_log1p', 'media_salience_volume_z',
    'afd_poll_support_z',
    'raw_article_count', 'download_fraction', 'kept_article_count',
    'cleaned_sample_size', 'cleaned_sample_fraction',
    'retrieval_mismatch_flag', 'high_download_sample_flag',
    'source'
]

existing_front = [c for c in preferred_front if c in panel.columns]
other_cols = [c for c in panel.columns if c not in existing_front]
panel_out = panel[existing_front + other_cols].copy()

panel_out.to_csv(PANEL_CSV_PATH, index=False)
panel_out.to_parquet(PANEL_PARQUET_PATH, index=False)

print('Saved panel CSV:', PANEL_CSV_PATH)
print('Saved panel parquet:', PANEL_PARQUET_PATH)
print('Final panel shape:', panel_out.shape)
print('Final panel columns:', list(panel_out.columns))
```

```
Saved panel CSV: c:\PythonProjects\AfD-
project\data\panel\monthly_salience_polling_panel.csv
Saved panel parquet: c:\PythonProjects\AfD-
project\data\panel\monthly_salience_polling_panel.parquet
Final panel shape: (108, 34)
Final panel columns: ['month', 'year_month', 'month_dt', 'year', 'month_num',
'afd_poll_support_mean', 'afd_poll_support_median', 'afd_poll_support_sd',
'n_polls', 'n_pollsters', 'nexis_total_results', 'media_salience_volume_raw',
'media_salience_volume_log1p', 'media_salience_volume_z', 'afd_poll_support_z',
'raw_article_count', 'download_fraction', 'kept_article_count',
```

```
'cleaned_sample_size', 'cleaned_sample_fraction', 'retrieval_mismatch_flag',  
'high_download_sample_flag', 'source', 'afd_poll_support_min',  
'afd_poll_support_max', 'first_poll_date', 'last_poll_date', 'month_media',  
'year_media', 'month_num_media', 'cleaned_sample_log1p', 'cleaned_sample_z',  
'month_dt_media', 'month_index']
```

1.1 Interpretation Notes

- This dataset is a descriptive merged monthly panel.
- Media salience is based on Nexis monthly total results (volume measure).
- Polling outcome is DAWUM monthly AfD Bundestag polling support.
- DAWUM coverage starts in 2017, so Cologne/2016 is outside this DAWUM-only merged panel.
- Chemnitz (2018-08) and Correctiv (2024-01) are inside the panel and included as visual reference lines.
- No causal claims are made in this notebook.
- Next step is descriptive pattern inspection; only after that should lag diagnostics or event-window setups be considered.